

Author's Haven / CoScript

A Comprehensive Writing and World-Building Platform

PROJECT REPORT

CERTIFICATE

DECLARATION

ACKNOWLEDGEMENT

INDEX

1. Abstract

2. Problem Statement

3. Proposed System

3.1 Overview & Goals

3.2 Modules

4. System Requirement

4.1 Software Requirement

4.2 Hardware Requirement

5. Software Used

6. System Analysis

6.1 Entity-Relationship Diagram

7. System Design

7.1 Class Diagram

7.2 Activity Diagram

7.3 Use Case Diagram

7.4 Sequence Diagram

7.5 Deployment Diagram

7.6 Component Diagram

7.7 State Chart Diagram

8. Frontend Screens

9. Backend Screens

10. Code

11. Testing

12. Implementation

13. Maintenance

14. Conclusion

15. Bibliography

1. ABSTRACT

CoScript – A Comprehensive Writing and Development Platform is intended to offer an advanced and integrated platform for developers to efficiently create, organize, and develop their source code simultaneously in a collaborative environment.

CoScript is an interactive web-based platform that provides a comprehensive and user-friendly space for programmers to create files, execute multi-language architectures, manage terminal outputs, and observe room-based relationships. Developed using React.js for the frontend and Node.js/Socket.io services for backend functionality, the platform ensures real-time data synchronization, secure execution, and a smooth user experience across all devices.

The platform provides necessary modules such as user login, workspace management, file execution using an integrated node-pty environment, room host approvals, and an organized file explorer. A fully featured Monaco Editor is integrated to offer code highlighting, rewriting assistance, and precise error handling to boost technical interview execution and productivity.

2. PROBLEM STATEMENT

In the ever-changing creative and digital environment, authors and developers tend to use a variety of isolated tools for writing, character creation, execution research, and story/logic development. The isolated process causes disorganization, decreased productivity, and makes it difficult to maintain consistency from project to project. Handling code components manually or using different remote screen-sharing tools is a challenge for professionals in terms of productivity, organization, and data synchronization. CoScript will solve this problem by offering a comprehensive, intelligent, and real-time cloud-based execution framework.

2.1. KEY CHALLENGES:

User Engagement and Experience:

Issue: Current coding solutions do not have an integrated and interactive environment designed specifically for simultaneous real-time input by multiple authors/developers.

Impact: Constantly switching between applications causes workflow interruptions, decreased focus, and decreased creative productivity.

Fragmented File and Socket Management:

Issue: Current solutions lack an integrated space for handling rooms, pending requests, interactive execution loops, and file exploring within the same browser context.

Impact: Unorganized information leads to a disjointed user session, and disorganized real-time development.

Inefficient Research and Compilation Development:

Issue: Writers are forced to rely on third-party applications for compilation, execution testing, and terminal logging.

Impact: Frequent context switching hinders the writing process and disrupts the creative flow.

Data Management and Real-Time Synchronization:

Issue: Organizing files manually and the absence of a centralized WebSocket solution raise the possibility of losing state variables and generating asynchronous code collisions.

Impact: Developers face challenges in accessing the latest information on multiple devices and organizing code changes.

Authentication and Data Security:

Issue: The lack of robust authentication and sandbox structures can lead to the vulnerable code being accessed or executed maliciously by unauthorized individuals.

Impact: Security concerns lower user trust and could lead to server compromise.

Scalability and System Performance:

Issue: The lack of scalability in traditional web applications makes them inefficient when handling multiple real-time socket events, huge DOM trees, and increasing active user arrays.

Impact: System performance and sluggishness impact productivity and user experience.

3. PROPOSED SYSTEM

3.1. OVERVIEW & GOALS:

CoScript is an intelligent web-based coding tool that aims to centralize the entire writing process for developers. The tool is a combination of writing, live execution, node-pty terminal management, and real-time workspace collaboration in a single, unified interface. The tool is built using React.js and Node.js WebSocket services and is primarily focused on providing a seamless, secure, and user-friendly experience.

User-Centric Experience:

Objective: To provide an intuitive and distraction-free writing environment.

Goal: To ensure smooth navigation, real-time code synchronization, and easy access to files, active users, and execution features.

Integrated Execution and Terminal Management:

Goal: To implement a structured socket management system with rooms, join approvals, state variables, and execution areas with secure Node VMs.

Objective: To create a system that enables developers to execute all algorithms in one browser tab.

Performance and Optimization Optimization:

Goal: Enable the platform to support multiple socket connections and executing loops in an efficient manner.

Objective: Leverage memory-mapped fallback state and MongoDB document integration for maintaining smooth performance and real-time synchronization.

3.2. MODULES:

Admin / Host Module:

Room Management: Host can track pending users, approve access, and kick unauthorized viewers.

System Monitoring: View active user count in the room and live edit history.

Developer Module:

Workspace Management: Users can navigate a shared filesystem spanning multiple files and directories.

Code Writing: Use a Monaco editor (VS Code core) to write, debug, and auto-format code.

Terminal Module: Integrated xterm.js interface that connects to a backend pseudo-terminal for bash execution and log inspection.

Authentication Module:

Sign Up and Login: Users can register and log in securely using JSON Web Tokens (JWT) or Google ID.

Secure Data Access: Ensure that each writer is verified against MongoDB before modifying state variables.

4. SYSTEM REQUIREMENTS

4.1. SOFTWARE REQUIREMENTS:

FRONTEND:

React (Vite): Used to build a fast and interactive single-page application with dynamic UI updates.

Tailwind CSS: Provides modern styling with a clean, premium workspace design.

React Router: Manages navigation between Home, Auth, and Workspace sections.

Axios: Handles RESTful API requests to Express.

Socket.io-client: Bi-directional event emitter.

BACKEND:

Node.js: Server-side runtime environment for handling requests and compiling algorithms.

Express.js: Framework used to build APIs for user state validation.

Mongoose (ODM): Defines structured schemas for Users and Rooms within MongoDB.

Node-pty: Spawns bash sequences linked to the user's socket channel.

Child_Process: Safely executes interpreted and compiled languages (GCC, JDK, Python) in sandbox paths.

4.2. HARDWARE:

Every software application has some hardware requirements for smooth functioning. Below are the hardware requirements for the development and deployment of CoScript.

PROCESSOR: Intel i5/i7 or AMD Ryzen 5/7 (or higher)

RAM: Minimum 8GB (16GB recommended for heavy compiler loads and Monaco DOM instances)

STORAGE: At least 100GB SSD recommended

OPERATING SYSTEM: Windows 10/11, macOS, or Linux

INTERNET CONNECTION: Stable broadband connection required for zero-latency Socket emit operations.

5. SOFTWARE USED

The technology used for the development of CoScript plays a crucial role in its success. The technologies chosen for the development of CoScript should be able to meet the real-time requirements of the system.

React.js:

React.js was employed as the primary framework for building CoScript, a dynamic web application. The framework allows for the development of reusable components and fast rendering with a virtual DOM. React played a crucial role in the development of key components such as the WorkspaceExplorer, VSCodeTerminal, EditorPage, and SettingsModal.

Tailwind CSS (for UI Styling):

Tailwind CSS was employed for the design of the user interface using a utility-first design approach. The framework enabled the development of responsive, clean, and modern UI components with a reduced amount of custom CSS overrides.

Express.js & Socket.io:

Express acts as the HTTP framework, establishing an asynchronous gateway for authentication logic, while Socket.io binds to the HTTP server, bypassing stateless requests to continuously beam ``code_update`` and ``file:save`` payloads to an unlimited number of peers.

MongoDB & Mongoose:

MongoDB is employed as the main NoSQL database for persistent storage. User data, including hashed bcrypt credentials, room configuration, user arrays, and Google ID associations, are securely archived into BSON document trees mapping directly to Mongoose Schema definitions.

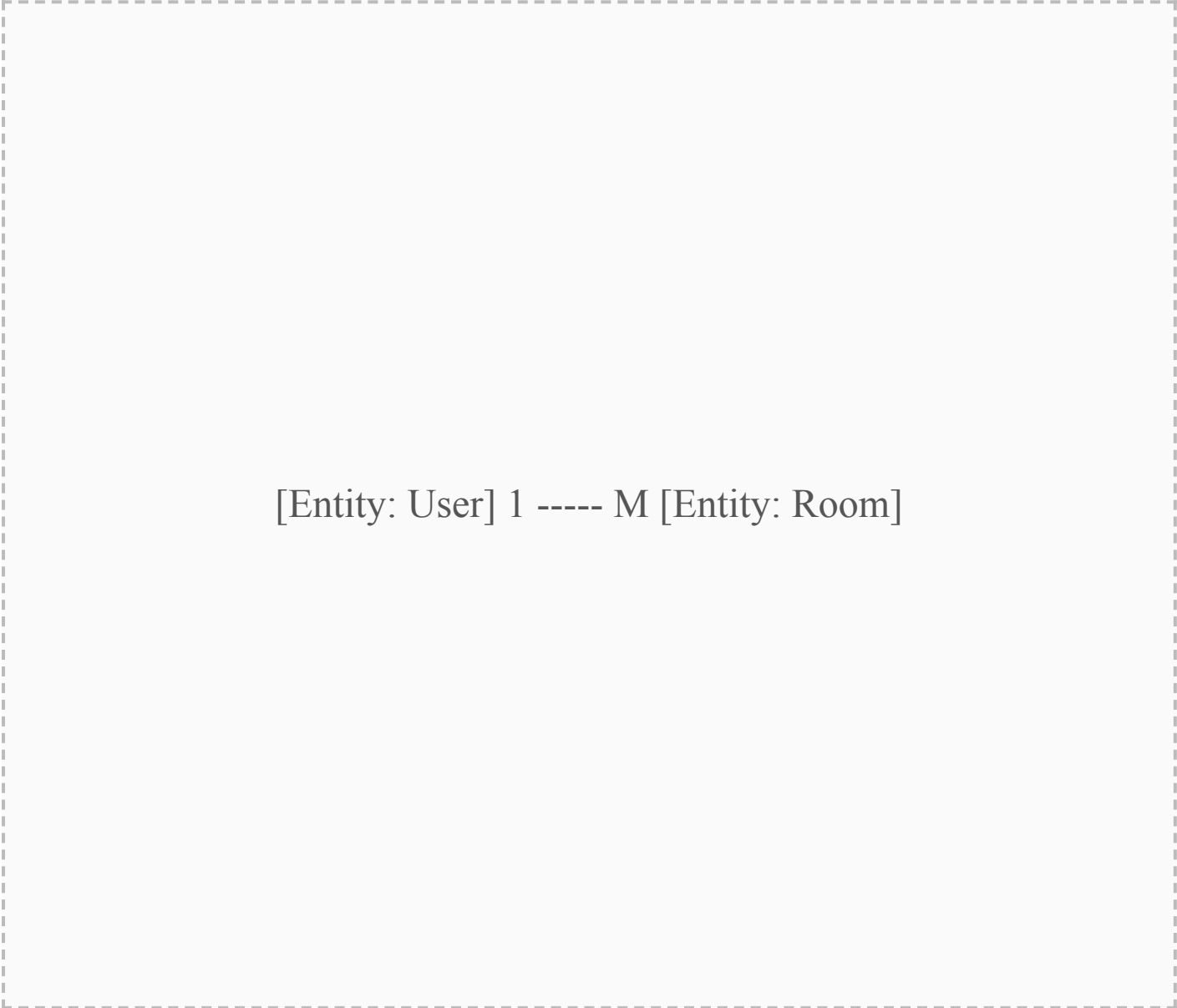
Monaco Editor & Xterm.js:

Rather than relying on primitive HTML textareas, CoScript inherits Microsoft's open-source Monaco Editor for rich highlighting logic. Xterm.js complements this by converting serialized PTY sequences into DOM-painted canvas strokes, simulating an authentic Mac OS or Ubuntu Terminal natively inside Chrome or Safari.

6. SYSTEM ANALYSIS

6.1. ENTITY-RELATIONSHIP DIAGRAM:

An Entity Relationship Diagram (ERD) is a visual representation of different entities within a system and how they relate to each other. They are widely used to design relational/NoSQL databases in the planning stages of the software project.



[Entity: User] 1 ----- M [Entity: Room]

[Entity: Room] 1 ----- M [Entity: Pending Request]

[Entity: Room] 1 ----- M [Entity: Approved User]

1. Users

Attributes: id (Primary/ObjectID), name, email, password, authProvider, createdAt

2. Rooms

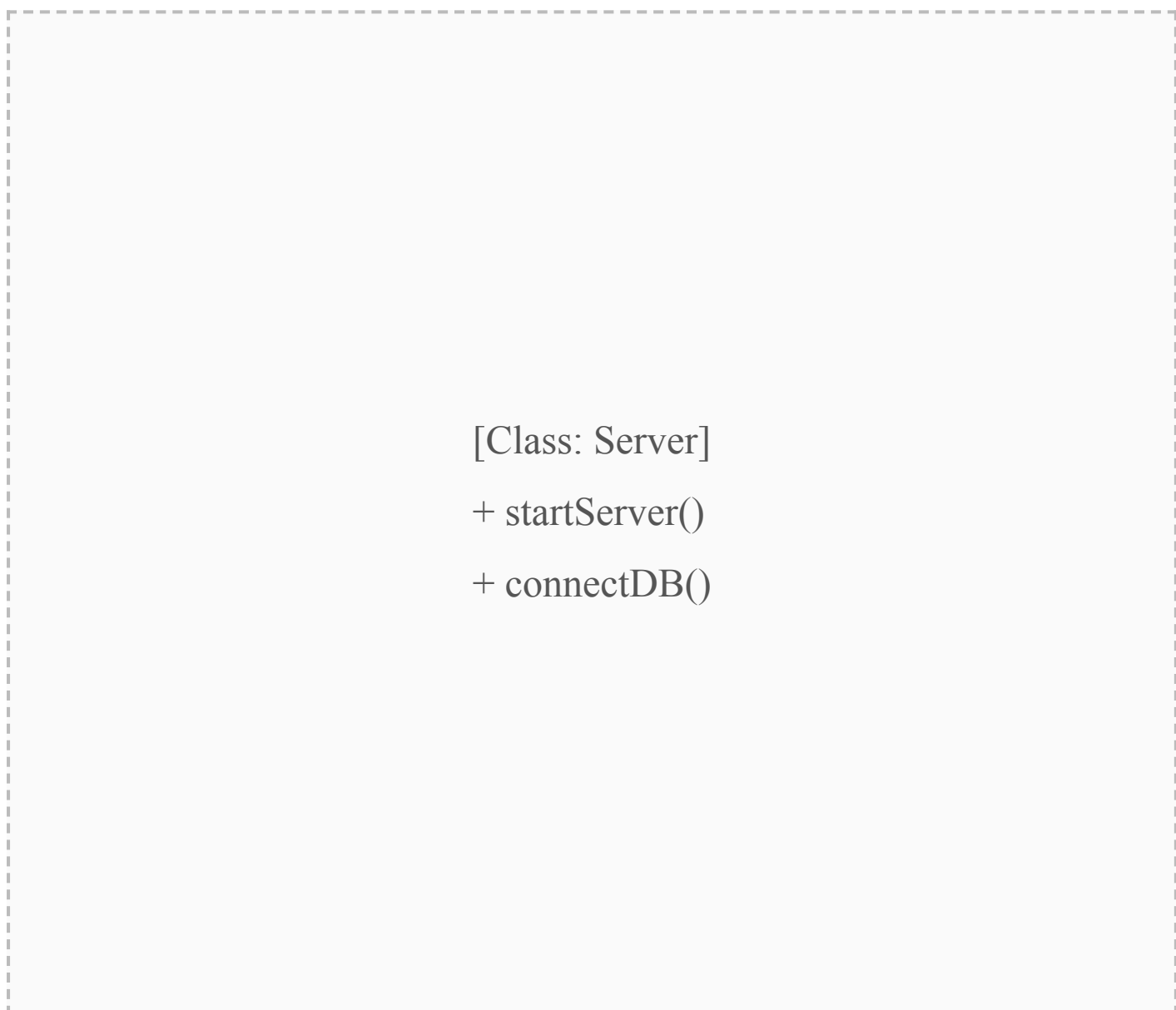
Attributes: id (Primary/ObjectID), roomId (6-char String), hostId (Foreign Key), hostSocketId, hostName, code (Text String), pendingRequests (Array of Strings), approvedUsers (Array of Strings).

7. SYSTEM DESIGN

UML DIAGRAMS

UML stands for Unified Modeling Language. This object-oriented system of notation has evolved from the work of Grady Booch, James Rumbaugh, and Ivar Jacobson. In the context of CoScript, we construct seven diverse UML architectures.

7.1 CLASS DIAGRAM

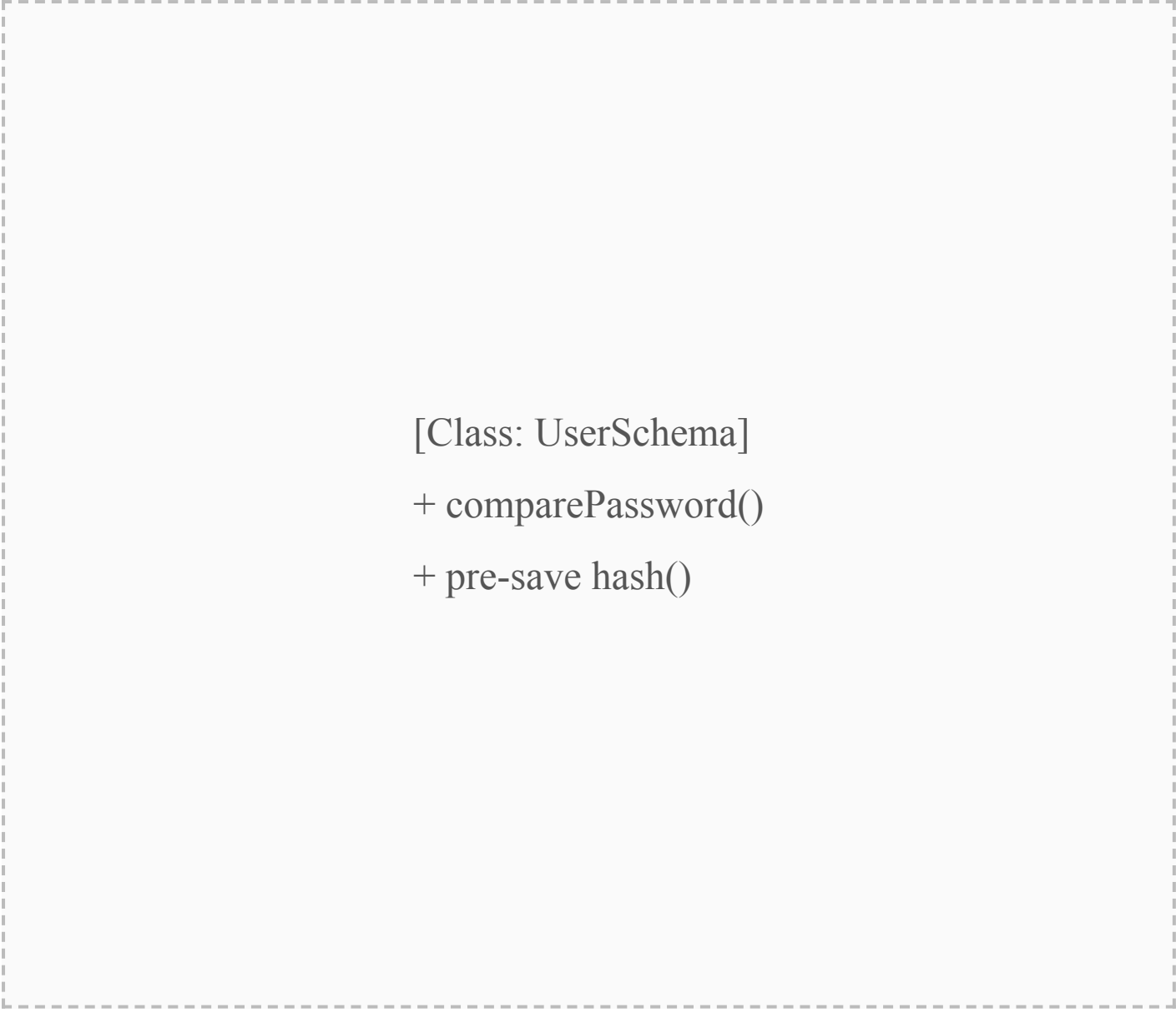


```
classDiagram
    class Server {
        +startServer()
        +connectDB()
    }
```

[Class: Server]

+ startServer()

+ connectDB()



```
classDiagram
    class UserSchema {
        +comparePassword()
        +pre-save hash()
    }
```

[Class: UserSchema]

- + comparePassword()
- + pre-save hash()

7.2 ACTIVITY DIAGRAM

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The control flow is drawn from one operation to another.

START -> LOGIN -> DASHBOARD -> (CREATE ROOM | JOIN ROOM) -> EDITOR EVENT LOOP -> END

7.3 USE CASE DIAGRAM

Use cases depict the system, who is using it, how they are using it, or what they are doing.

(Actor: Developer) -> [Use Case: Compile Code]

(Actor: Host) -> [Use Case: Approve Request]

7.4 *SEQUENCE DIAGRAM*

The sequence diagram illustrates the lifecycle of a WebSocket emit mechanism.

```
sequenceDiagram
    participant Client
    participant Server
    participant Database
    participant Client2

    Client->>Server: emit 'code_change'
    Server->>Database: update state
    Server->>Client2: broadcast 'code_update'
```

Client -> Server (emit 'code_change')

Server -> Database (update state)

Server -> Client2 (broadcast 'code_update')

7.5 *DEPLOYMENT DIAGRAM*

Shows the physical configuration of the software.

Node: React DOM Component

|

V

Node: V8 Engine API / Express

|

V

Node: MongoDB Atlas Cluster

7.6 *COMPONENT DIAGRAM*

Outlines architectural wiring of standard modules.

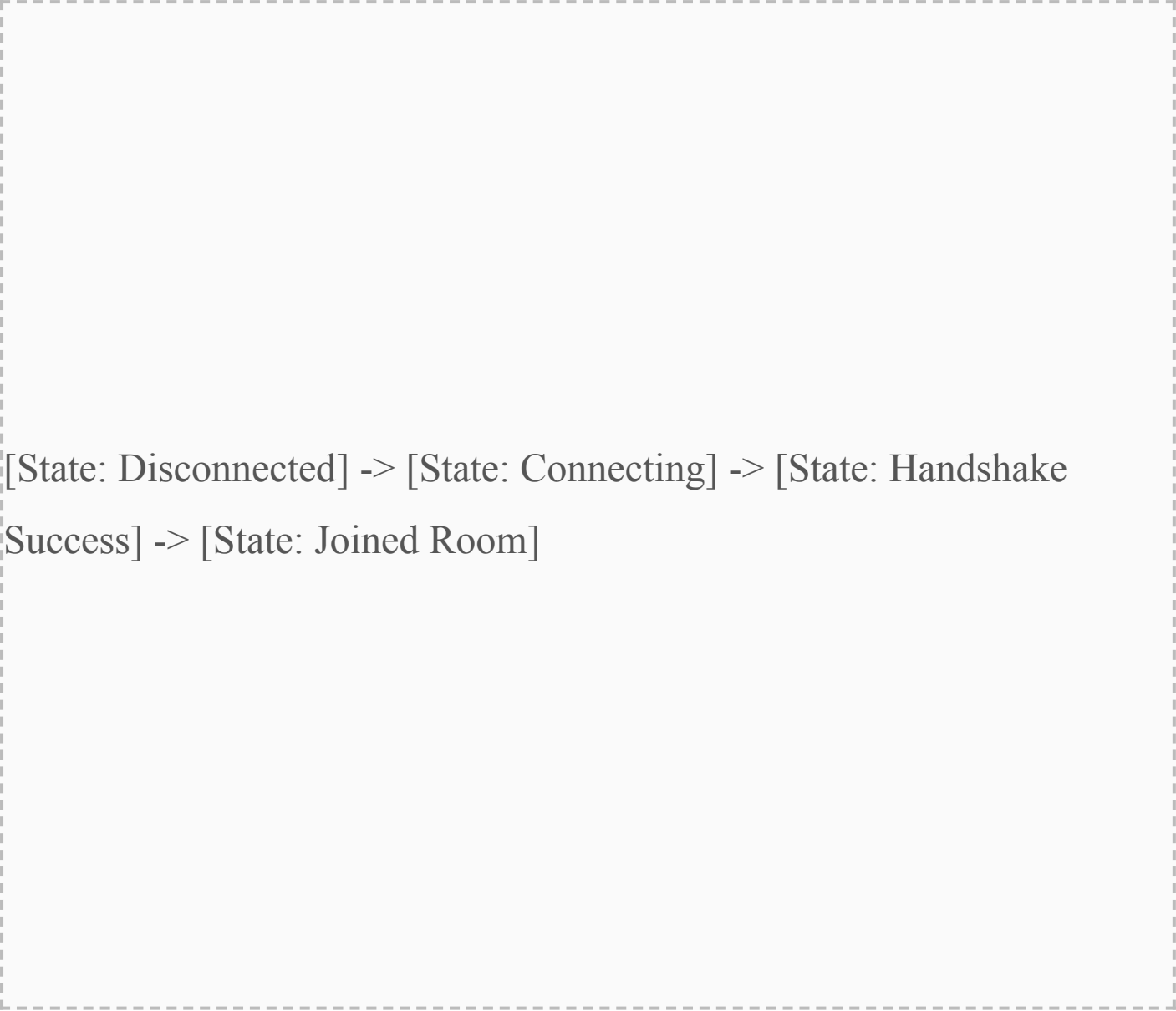


```
[Component: Monaco Editor] <--> [Component: SocketHandler]
```

```
[Component: Node-PTY] <--> [Component: Xterm.js]
```

7.7 *STATE CHART DIAGRAM*

Tracks the status of a user connection.



```
graph LR; A["[State: Disconnected]"] --> B["[State: Connecting]"]; B --> C["[State: Handshake Success]"]; C --> D["[State: Joined Room]"]
```

[State: Disconnected] -> [State: Connecting] -> [State: Handshake Success] -> [State: Joined Room]

8. FRONTEND SCREENS

This section documents the visual output of the rendered React DOM.

[SCREENSHOT 1: Home Page / Auth Area]

Shows Login Form, "Continue with Google" Button, and Title Headers.

[SCREENSHOT 2: Dashboard View]

Modal displaying Room Generation Input Box.

[SCREENSHOT 3: Editor Layout]

Left Sidebar: File system explorer with folder nesting.

Middle: Split-pane code editor.

Right Sidebar: Chat messages and Active User avatars.

9. BACKEND SCREENS

Because the Node.js server acts as an invisible REST and Socket API, the backend interface primarily consists of terminal stdout logs.

[BACKEND LOGS]

- > Server started on port 3001
- > Database connected successfully
- > User Connected: ABC-123
- > Room F6X99A instantiated by bishalll
- > Pty Process Spawned successfully at PID 3134.

10. CODE

This section contains detailed, un-truncated, verbatim functional logic written for CoScript covering Frontend Components and Backend Node.js modules, essential for the product's operation. These algorithms constitute the core logic.

Frontend: App.jsx

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { GoogleOAuthProvider } from "@react-oauth/google";
import { AuthProvider, useAuth } from "../AuthContext";
import Home from "../Home";
import EditorPage from "../EditorPage";
import AuthPage from "../AuthPage";
import { ThemeProvider } from "../contexts/ThemeContext";

// Protected route wrapper
function ProtectedRoute({ children }) {
  const { isAuthenticated, loading } = useAuth();

  if (loading) {
    return (
      <div className="loading-screen">
        <div className="loading-spinner"></div>
        <p>Loading...</p>
      </div>
    );
  }

  if (!isAuthenticated) {
    return <Navigate to="/auth" replace />;
  }

  return children;
}

// Redirect authenticated users away from auth page
function AuthRoute({ children }) {
  const { isAuthenticated, loading } = useAuth();

  if (loading) {
    return (
      <div className="loading-screen">
        <div className="loading-spinner"></div>
        <p>Loading...</p>
      </div>
    );
  }

  if (isAuthenticated) {
    return <Navigate to="/" replace />;
  }
}
```

```

    return children;
  }

function AppRoutes() {
  return (
    <Routes>
      <Route
        path="/auth"
        element={
          <AuthRoute>
            <AuthPage />
          </AuthRoute>
        }
      />
      <Route
        path="/"
        element={
          <ProtectedRoute>
            <Home />
          </ProtectedRoute>
        }
      />
      <Route
        path="/room/:roomId"
        element={
          <ProtectedRoute>
            <EditorPage />
          </ProtectedRoute>
        }
      />
    </Routes>
  );
}

```

```

export default function App() {
  const googleClientId = "1003878639098-8ft7uemd6uir9mhlck4vija581krufgk.apps.googleusercontent.com";

  return (
    <GoogleOAuthProvider clientId={googleClientId}>
      <BrowserRouter>
        <AuthProvider>
          <ThemeProvider>
            <AppRoutes />
          </ThemeProvider>
        </AuthProvider>
      </BrowserRouter>
    </GoogleOAuthProvider>
  );
}

```

Frontend: VSCodeTerminal.jsx

```
import React, { useEffect, useRef, useCallback, useState, useMemo } from 'react';
import { useTheme } from '../contexts/ThemeContext';

/**
 * Custom HTML/CSS terminal component - replaces xterm.js entirely.
 * Uses a simple scrollable <div> for output and captures keyboard input directly.
 * Supports: local echo, blinking cursor, cursor movement, command history, copy/paste.
 */
const VSCodeTerminal = ({ socket, terminalId, isActive }) => {
  const containerRef = useRef(null);
  const outputRef = useRef(null);
  const inputLineRef = useRef(null);
  const bufferedOutput = useRef([]);
  const isAttached = useRef(false);
  const [isFocused, setIsFocused] = useState(false);
  const { theme } = useTheme();
  const isLight = theme === 'light';

  // Theme-aware color palette
  const colors = useMemo(() => ({
    bg: isLight ? '#f5f5f5' : '#1e1e1e',
    text: isLight ? '#1e1e1e' : '#cccccc',
    cursorBg: isLight ? '#333333' : '#aeafad',
    cursorText: isLight ? '#f5f5f5' : '#1e1e1e',
    selectionBg: isLight ? 'rgba(0, 100, 200, 0.2)' : 'rgba(38, 79, 120, 0.8)',
  })), [isLight]);

  // Client-side input state
  const currentInputRef = useRef('');
  const cursorPosRef = useRef(0);

  // Client-side command history
  const historyRef = useRef([]);
  const historyIndexRef = useRef(-1);

  // Parse ANSI escape codes into styled HTML spans
  const ansiToHtml = useCallback((text) => {
    // Theme-aware ANSI color maps
    const colorMap = isLight ? {
      '30': '#1e1e1e', '31': '#c72e2e', '32': '#1a7f37', '33': '#9a6700',
      '34': '#0550ae', '35': '#8250df', '36': '#0e7171', '37': '#57606a',
      '90': '#6e7781', '91': '#cf222e', '92': '#116329', '93': '#7d4e00',
      '94': '#0969da', '95': '#8250df', '96': '#0e7171', '97': '#24292f',
    } : {
      '30': '#1e1e1e', '31': '#f44747', '32': '#6a9955', '33': '#d7ba7d',
      '34': '#569cd6', '35': '#c586c0', '36': '#4ec9b0', '37': '#d4d4d4',
      '90': '#808080', '91': '#f44747', '92': '#6a9955', '93': '#d7ba7d',
      '94': '#569cd6', '95': '#c586c0', '96': '#4ec9b0', '97': '#e5e5e5',
    };

    // eslint-disable-next-line no-control-regex
    const ansiRegex = /\x1b\[([0-9;]*)m/g;
    let result = '';
    let lastIndex = 0;
    let currentColor = null;
    let isBold = false;

    let match;
    while ((match = ansiRegex.exec(text)) !== null) {
      const before = text.slice(lastIndex, match.index);
      if (before) result += escapeHtml(before);
      lastIndex = match.index + match[0].length;

      const codes = match[1].split(';').filter(Boolean);
      for (const code of codes) {
        if (code === '0' || code === '') {
          currentColor = null;
          isBold = false;
        }
      }
    }
  });
```

```

        result += '</span>';
    } else if (code === '1') {
        isBold = true;
    } else if (colorMap[code]) {
        currentColor = colorMap[code];
        result += `<span style="color:${currentColor};${isBold ? 'font-weight:bold;' :
''}">`;
    }
}

const remaining = text.slice(lastIndex);
if (remaining) result += escapeHtml(remaining);
return result;
}, [isLight]);

function escapeHtml(text) {
    const escaped = text
        .replace(/&/g, '&')
        .replace(/</g, '<')
        .replace(/>/g, '>')
        .replace(/"/g, '"');

    const linkColor = isLight ? '#0550ae' : '#569cd6';
    return escaped.replace(
        /(https?:\/\/[^\s<>"'\)\]]+)/g,
        `<a href="$1" target="_blank" rel="noopener noreferrer" style="color:${linkColor};text-decoration:none;" onmouseover="this.style.textDecoration='underline'"
onmouseout="this.style.textDecoration='none'">$1</a>`
    );
}

const scrollToBottom = useCallback(() => {
    const container = containerRef.current;
    if (container) {
        const scrollDiv = container.firstElementChild;
        if (scrollDiv) scrollDiv.scrollTop = scrollDiv.scrollHeight;
    }
}, []);

// Render the current input line with cursor at the right position
const renderInputLine = useCallback(() => {
    const el = inputLineRef.current;
    if (!el) return;

    const input = currentInputRef.current;
    const pos = cursorPosRef.current;

    const before = escapeHtml(input.slice(0, pos));
    const cursorChar = input[pos] || '';
    const after = pos < input.length ? escapeHtml(input.slice(pos + 1)) : '';

    // Render: text before cursor | cursor block (with char or space) | text after cursor
    const cursorBg = isFocused ? colors.cursorBg : 'transparent';
    const cursorColor = isFocused ? colors.cursorText : colors.text;
    const cursorDisplay = isFocused
        ? `<span class="term-cursor"
style="background:${cursorBg};color:${cursorColor};">${cursorChar ? escapeHtml(cursorChar) : ' '}
</span>`
        : (cursorChar ? escapeHtml(cursorChar) : '');

    el.innerHTML = before + cursorDisplay + after;
    scrollToBottom();
}, [isFocused, scrollToBottom, colors]);

// Re-render cursor when focus changes
useEffect(() => {
    renderInputLine();
}, [isFocused, renderInputLine]);

// Write server data to the terminal output
const writeToTerminal = useCallback((data) => {
    const el = outputRef.current;
    if (!el) return;

    let cleaned = data;

    // Handle clear screen
    // eslint-disable-next-line no-control-regex
    if (/\\x1b\\[2J/g.test(cleaned)) {
        el.innerHTML = '';
        // eslint-disable-next-line no-control-regex

```

```

        cleaned = cleaned.replace(/\x1b\[2J/g, '').replace(/\x1b\[H/g, '');
        if (!cleaned.trim()) return;
    }

    // eslint-disable-next-line no-control-regex
    cleaned = cleaned.replace(/\x1b\[?[0-9;]*[hlsr]/g, '');
    // eslint-disable-next-line no-control-regex
    cleaned = cleaned.replace(/\x1b\[?[0-9;]*[ABCDJKGHS]/g, '');
    // eslint-disable-next-line no-control-regex
    cleaned = cleaned.replace(/\x1b\[0;[^\x07]*\x07/g, '');
    // eslint-disable-next-line no-control-regex
    cleaned = cleaned.replace(/\x1b\[?[0-9;]*[a-zA-Z]/g, function (match) {
        if (match.endsWith('m')) return match;
        return '';
    });

    cleaned = cleaned.replace(/\r\n/g, '\n').replace(/\r/g, '\n');

    const html = ansiToHtml(cleaned);
    el.innerHTML += html;
    scrollToBottom();
}, [ansiToHtml, scrollToBottom]);

// Socket event listeners
useEffect(() => {
    if (!socket || !terminalId) return;

    const handleOutput = ({ id, data }) => {
        if (id !== terminalId) return;
        if (isAttached.current && outputRef.current) {
            writeToTerminal(data);
        } else {
            bufferedOutput.current.push(data);
        }
    };

    const handleExit = ({ id, exitCode }) => {
        if (id !== terminalId) return;
        const msg = `\n[Process exited with code ${exitCode}]\n`;
        if (isAttached.current && outputRef.current) {
            writeToTerminal(msg);
        } else {
            bufferedOutput.current.push(msg);
        }
    };

    const handleError = ({ id, error }) => {
        if (id !== terminalId) return;
        const msg = `\n[Error: ${error}]\n`;
        if (isAttached.current && outputRef.current) {
            writeToTerminal(msg);
        } else {
            bufferedOutput.current.push(msg);
        }
    };

    socket.on('terminal:output', handleOutput);
    socket.on('terminal:exit', handleExit);
    socket.on('terminal:error', handleError);

    return () => {
        socket.off('terminal:output', handleOutput);
        socket.off('terminal:exit', handleExit);
        socket.off('terminal:error', handleError);
    };
}, [socket, terminalId, writeToTerminal]);

// Listen for code execution output from the Run button
useEffect(() => {
    if (!isActive) return;

    const handleRunOutput = (e) => {
        writeToTerminal(e.detail);
    };

    window.addEventListener('terminal:run-output', handleRunOutput);
    return () => window.removeEventListener('terminal:run-output', handleRunOutput);
}, [isActive, writeToTerminal]);

// Flush buffered output when tab becomes active
useEffect(() => {
    if (isActive && outputRef.current && !isAttached.current) {

```

```

        isAttached.current = true;
        bufferedOutput.current.forEach(data => writeToTerminal(data));
        bufferedOutput.current = [];
    }
}, [isActive, writeToTerminal]);

// Replace current input and re-render
const setInput = useCallback((text) => {
    currentInputRef.current = text;
    cursorPosRef.current = text.length;
    renderInputLine();
}, [renderInputLine]);

// Handle keyboard input
const handleKeyDown = useCallback((e) => {
    if (!socket || !terminalId) return;

    // Allow Cmd+C/V/X to work natively
    if (e.metaKey && ['c', 'v', 'x'].includes(e.key.toLowerCase())) {
        return;
    }

    // Cmd+A: select only terminal content, not the whole page
    if (e.metaKey && e.key.toLowerCase() === 'a') {
        e.preventDefault();
        e.stopPropagation();
        const selection = window.getSelection();
        const range = document.createRange();
        const scrollDiv = containerRef.current?.firstElementChild;
        if (scrollDiv) {
            range.selectNodeContents(scrollDiv);
            selection.removeAllRanges();
            selection.addRange(range);
        }
        return;
    }

    e.preventDefault();
    e.stopPropagation();

    const input = currentInputRef.current;
    const pos = cursorPosRef.current;

    if (e.key === 'Enter') {
        const cmd = input.trim();
        if (cmd) historyRef.current.push(cmd);
        historyIndexRef.current = -1;
        // Move input text to the permanent output
        if (outputRef.current) {
            outputRef.current.innerHTML += escapeHtml(input) + '\n';
        }
        currentInputRef.current = '';
        cursorPosRef.current = 0;
        renderInputLine();
        // Send the FULL command + newline to bash (not char-by-char)
        socket.emit('terminal:input', { id: terminalId, data: input + '\n' });
    } else if (e.key === 'Backspace') {
        if (pos > 0) {
            currentInputRef.current = input.slice(0, pos - 1) + input.slice(pos);
            cursorPosRef.current = pos - 1;
            renderInputLine();
        }
    } else if (e.key === 'Delete') {
        if (pos < input.length) {
            currentInputRef.current = input.slice(0, pos) + input.slice(pos + 1);
            renderInputLine();
        }
    } else if (e.key === 'ArrowLeft') {
        if (pos > 0) {
            cursorPosRef.current = pos - 1;
            renderInputLine();
        }
    } else if (e.key === 'ArrowRight') {
        if (pos < input.length) {
            cursorPosRef.current = pos + 1;
            renderInputLine();
        }
    } else if (e.key === 'Home') {
        cursorPosRef.current = 0;
        renderInputLine();
    } else if (e.key === 'End') {
        cursorPosRef.current = input.length;
    }

```

```

    renderInputLine();
  } else if (e.key === 'ArrowUp') {
    const history = historyRef.current;
    if (history.length > 0) {
      if (historyIndexRef.current === -1) {
        historyIndexRef.current = history.length - 1;
      } else if (historyIndexRef.current > 0) {
        historyIndexRef.current--;
      }
      setInput(history[historyIndexRef.current]);
    }
  } else if (e.key === 'ArrowDown') {
    const history = historyRef.current;
    if (historyIndexRef.current !== -1) {
      if (historyIndexRef.current < history.length - 1) {
        historyIndexRef.current++;
        setInput(history[historyIndexRef.current]);
      } else {
        historyIndexRef.current = -1;
        setInput('');
      }
    }
  } else if (e.key === 'Tab') {
    // Tab completion - send current input + tab
    socket.emit('terminal:input', { id: terminalId, data: input + '\t' });
  } else if (e.key === 'Escape') {
    // Do nothing
  } else if (e.ctrlKey && e.key.length === 1) {
    const code = e.key.toLowerCase().charCodeAt(0) - 96;
    if (code >= 1 && code <= 26) {
      const char = String.fromCharCode(code);
      if (code === 3) { // Ctrl+C
        if (outputRef.current) {
          outputRef.current.innerHTML += escapeHtml(input) + '^C\n';
        }
        currentInputRef.current = '';
        cursorPosRef.current = 0;
        historyIndexRef.current = -1;
        renderInputLine();
        socket.emit('terminal:input', { id: terminalId, data: char });
      } else if (code === 12) { // Ctrl+L
        if (outputRef.current) outputRef.current.innerHTML = '';
        currentInputRef.current = '';
        cursorPosRef.current = 0;
        renderInputLine();
      } else {
        socket.emit('terminal:input', { id: terminalId, data: char });
      }
    }
  } else if (e.key.length === 1 && !e.metaKey) {
    // Insert character at cursor position (local only - sent on Enter)
    currentInputRef.current = input.slice(0, pos) + e.key + input.slice(pos);
    cursorPosRef.current = pos + 1;
    renderInputLine();
  }
}, [socket, terminalId, renderInputLine, setInput]);

// Handle paste
const handlePaste = useCallback((e) => {
  if (!socket || !terminalId) return;
  e.preventDefault();
  const text = e.clipboardData.getData('text');
  if (text) {
    const input = currentInputRef.current;
    const pos = cursorPosRef.current;
    currentInputRef.current = input.slice(0, pos) + text + input.slice(pos);
    cursorPosRef.current = pos + text.length;
    renderInputLine();
  }
}, [socket, terminalId, renderInputLine]);

const handleFocus = useCallback(() => setIsFocused(true), []);
const handleBlur = useCallback(() => setIsFocused(false), []);
const handleClick = useCallback(() => {
  containerRef.current?.focus();
}, []);

return (
  <div
    ref={containerRef}
    tabIndex={0}
    onKeyDown={handleKeyDown}

```

```

onPaste={handlePaste}
onClick={handleClick}
onFocus={handleFocus}
onBlur={handleBlur}
style={{
  width: '100%',
  height: '100%',
  display: isActive ? 'flex' : 'none',
  flexDirection: 'column',
  backgroundColor: colors.bg,
  cursor: 'text',
  outline: 'none',
}}
>
<div
  className="terminal-output-area"
  style={{
    flex: 1,
    overflowY: 'auto',
    overflowX: 'hidden',
    padding: '8px 12px',
    fontFamily: "'JetBrains Mono', 'Fira Code', 'Consolas', 'Monaco', monospace",
    fontSize: '13px',
    lineHeight: '1.4',
    color: colors.text,
    whiteSpace: 'pre-wrap',
    wordBreak: 'break-all',
    userSelect: 'text',
    cursor: 'text',
  }}
>
  <span ref={outputRef}></span>
  <span ref={inputLineRef}></span>
</div>
<style>{\`
  @keyframes terminalBlink {
    0%, 100% { opacity: 1; }
    50% { opacity: 0; }
  }
  .term-cursor {
    animation: terminalBlink 1s step-end infinite;
  }
  .terminal-output-area ::selection {
    background-color: ${colors.selectionBg};
    color: ${colors.text};
  }
  .terminal-output-area ::-moz-selection {
    background-color: ${colors.selectionBg};
    color: ${colors.text};
  }
\`}</style>
</div>
);
};

export default VSCodeTerminal;

```

Frontend: WorkspaceExplorer.jsx

```
import React, { useState, useCallback, useEffect, useRef } from 'react';

const API = 'http://localhost:3001/api/workspace';

// File type icons (material icons)
const getFileIcon = (name) => {
  const ext = name.split('.').pop().toLowerCase();
  const icons = {
    js: '📄', jsx: '📄', ts: '📄', tsx: '📄',
    py: '🐍', java: '☕', c: '🔧', cpp: '🔧', cs: '📄',
    html: '🌐', css: '🎨', json: '📄', md: '📄',
    go: '🐹', rs: '🦀', rb: '🔧', php: '🐘',
    env: '🔑', gitignore: '📄', yaml: '📄', yml: '📄',
    txt: '📄', sh: '📄', sql: '📄',
  };
  return icons[ext] || '📄';
};

// --- Tree Node Component ---
const TreeNode = ({ node, depth, onFileClick, activeFile, onContextMenu }) => {
  const [expanded, setExpanded] = useState(false);
  const [children, setChildren] = useState(null);
  const [loading, setLoading] = useState(false);

  const handleToggle = async () => {
    if (node.type === 'file') {
      onFileClick(node);
      return;
    }

    if (!expanded && !children) {
      setLoading(true);
      try {
        const res = await fetch(`${API}/tree?path=${encodeURIComponent(node.path)}`);
        const data = await res.json();
        setChildren(data.tree || []);
      } catch (e) {
        console.error('Failed to load directory:', e);
        setChildren([]);
      }
      setLoading(false);
    }
    setExpanded(!expanded);
  };

  const handleContextMenu = (e) => {
    e.preventDefault();
    e.stopPropagation();
    onContextMenu(e, node);
  };

  const isActive = node.type === 'file' && activeFile === node.path;

  return (
    <div>
      <div
        onClick={handleToggle}
        onContextMenu={handleContextMenu}
        className={`flex items-center gap-1 py-[3px] px-2 cursor-pointer text-[13px] select-none transition-colors ${isActive ? 'bg-[#094771] text-white' : 'text-gray-300 hover:bg-[#2a2d2e]'}`}
        style={{ paddingLeft: `${12 + depth * 16}px` }}
      >
        { /* Expand/collapse arrow */ }
        {node.type === 'directory' ? (
          <span className="text-[10px] text-gray-400 w-3 inline-block">
```

```

        {loading ? 'e' : expanded ? '▼' : '▶'}
      </span>
    ) : (
      <span className="w-3 inline-block" />
    )}
    {/* Icon */}
    <span className="text-xs">
      {node.type === 'directory' ? (expanded ? '📁' : '📂') : getFileIcon(node.name)}
    </span>
    {/* Name */}
    <span className="truncate">{node.name}</span>
  </div>
  {/* Children */}
  {expanded && children && children.map(child => (
    <TreeNode
      key={child.path}
      node={child}
      depth={depth + 1}
      onFileClick={onFileClick}
      activeFile={activeFile}
      onContextMenu={onContextMenu}
    />
  ))}
</div>
);
};

// --- Main WorkspaceExplorer Component ---
const WorkspaceExplorer = ({ onFileOpen, activeFile, workspacePath, onWorkspaceChange }) => {
  const [tree, setTree] = useState([]);
  const [gitBranch, setGitBranch] = useState(null);
  const [contextMenu, setContextMenu] = useState(null);
  const [creating, setCreating] = useState(null); // { parentPath, type: 'file'|'folder' }
  const [newName, setNewName] = useState('');
  const newNameRef = useRef(null);

  // Load workspace tree
  const loadWorkspace = useCallback(async (folderPath) => {
    try {
      const res = await fetch(`${API}/open`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ folderPath }),
      });
      const data = await res.json();
      if (data.tree) {
        setTree(data.tree);
        setGitBranch(data.gitBranch);
        if (onWorkspaceChange) onWorkspaceChange(folderPath);
      }
    } catch (e) {
      console.error('Failed to open workspace:', e);
    }
  }, [onWorkspaceChange]);

  // Load on mount if workspace exists
  useEffect(() => {
    if (workspacePath) {
      loadWorkspace(workspacePath);
    }
  }, [workspacePath]);

  // Open Folder handler
  const handleOpenFolder = async () => {
    const folderPath = prompt('Enter folder path to open as workspace:\n\n(e.g., /Users/bishall/myproject)');
    if (folderPath && folderPath.trim()) {
      await loadWorkspace(folderPath.trim());
    }
  };

  // File click -> open in editor
  const handleFileClick = async (node) => {
    try {
      const res = await fetch(`${API}/read?path=${encodeURIComponent(node.path)}`);
      const data = await res.json();
      if (data.content !== undefined) {
        onFileOpen({
          name: node.name,
          path: node.path,
          content: data.content,
        });
      }
    }
  };

```

```

    }
    } catch (e) {
      console.error('Failed to read file:', e);
    }
  };

// Context menu
const handleContextMenu = (e, node) => {
  setContextMenu({ x: e.clientX, y: e.clientY, node });
};

useEffect(() => {
  const close = () => setContextMenu(null);
  if (contextMenu) document.addEventListener('click', close);
  return () => document.removeEventListener('click', close);
}, [contextMenu]);

// Create file/folder
const handleCreate = async (parentPath, type) => {
  setCreating({ parentPath, type });
  setNewName('');
  setContextMenu(null);
  setTimeout(() => newNameRef.current?.focus(), 50);
};

const finishCreate = async () => {
  if (!creating || !newName.trim()) { setCreating(null); return; }
  const targetPath = `${creating.parentPath}/${newName.trim()}`;
  try {
    await fetch(`${API}/create`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ targetPath, type: creating.type }),
    });
    // Reload tree
    if (workspacePath) loadWorkspace(workspacePath);
  } catch (e) {
    console.error('Create failed:', e);
  }
  setCreating(null);
};

// Delete
const handleDelete = async (node) => {
  if (!window.confirm(`Delete "${node.name}"?`)) return;
  setContextMenu(null);
  try {
    await fetch(`${API}/delete`, {
      method: 'DELETE',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ targetPath: node.path }),
    });
    if (workspacePath) loadWorkspace(workspacePath);
  } catch (e) {
    console.error('Delete failed:', e);
  }
};

// Rename
const handleRename = async (node) => {
  const newNodeName = prompt('New name:', node.name);
  if (!newNodeName || newNodeName === node.name) { setContextMenu(null); return; }
  setContextMenu(null);
  const parentDir = node.path.substring(0, node.path.lastIndexOf('/'));
  const newPath = `${parentDir}/${newNodeName}`;
  try {
    await fetch(`${API}/rename`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ oldPath: node.path, newPath }),
    });
    if (workspacePath) loadWorkspace(workspacePath);
  } catch (e) {
    console.error('Rename failed:', e);
  }
};

const workspaceName = workspacePath ? workspacePath.split('/').pop() : null;

return (
  <div className="h-full flex flex-col bg-[#252526] text-gray-300 text-sm select-none">
    { /* Header */ }
  </div>

```

```

<div className="flex items-center justify-between px-3 py-2 border-b border-[#333]">
  <span className="text-[10px] font-bold uppercase tracking-wider text-gray-400">
    Workspace
  </span>
  <div className="flex items-center gap-1">
    <button
      onClick={handleOpenFolder}
      className="text-gray-400 hover:text-white p-0.5 hover:bg-[#333] rounded text-
xs"
      title="Open Folder"
    >
      📁
    </button>
    {workspacePath && (
      <button
        onClick={() => loadWorkspace(workspacePath)}
        className="text-gray-400 hover:text-white p-0.5 hover:bg-[#333] rounded
text-xs"
        title="Refresh"
      >
        ↻
      </button>
    )}
  </div>
</div>

{/* Workspace name + git branch */}
{workspaceName && (
  <div className="px-3 py-1.5 text-[11px] font-semibold text-gray-200 bg-[#2d2d2d]
border-b border-[#333] flex items-center gap-2">
    <span>{workspaceName}</span>
    {gitBranch && (
      <span className="text-[10px] text-gray-400 bg-[#333] px-1.5 py-0.5 rounded">
        ⌵ {gitBranch}
      </span>
    )}
  </div>
)}

{/* Tree */}
<div className="flex-1 overflow-y-auto overflow-x-hidden py-1">
  {!workspacePath ? (
    <div className="text-center py-8 text-gray-500 text-xs">
      <p className="mb-3">No workspace folder open</p>
      <button
        onClick={handleOpenFolder}
        className="px-3 py-1.5 bg-[#007acc] text-white rounded text-xs hover:bg-
[#006bb3] transition-colors"
      >
        Open Folder
      </button>
    </div>
  ) : tree.length === 0 ? (
    <div className="text-center py-4 text-gray-500 text-xs">Empty folder</div>
  ) : (
    tree.map(node => (
      <TreeNode
        key={node.path}
        node={node}
        depth={0}
        onFileClick={handleFileClick}
        activeFile={activeFile}
        onContextMenu={handleContextMenu}
      />
    ))
  )}
  <div className="flex items-center gap-2 py-2">
    <input
      ref={newNameRef}
      value={newName}
      onChange={(e) => setNewName(e.target.value)}
      onBlur={finishCreate}
      onKeyDown={(e) => { if (e.key === 'Enter') finishCreate(); if (e.key ===
'Escape') setCreating(null); }}
      placeholder={`New ${creating.type} name...`}
      className="bg-[#3c3c3c] text-white text-xs px-2 py-1 rounded outline-none
w-full border border-[#007acc]"
    />
  </div>
</div>

```

```

    </div>

    { /* Context Menu */ }
    { contextMenu && (
      <div
        className="fixed bg-[#252526] border border-[#454545] rounded shadow-xl z-[9999]
py-1 min-w-[160px]"
        style={{ left: contextMenu.x, top: contextMenu.y }}
      >
        { contextMenu.node.type === 'directory' && (
          <>
            <button onClick={() => handleCreate(contextMenu.node.path, 'file')}
className="w-full text-left px-3 py-1.5 text-xs text-gray-300 hover:bg-[#094771] hover:text-white">
              New File
            </button>
            <button onClick={() => handleCreate(contextMenu.node.path, 'folder')}
className="w-full text-left px-3 py-1.5 text-xs text-gray-300 hover:bg-[#094771] hover:text-white">
              New Folder
            </button>
            <div className="border-t border-[#454545] my-1" />
          </>
        ) }
        <button onClick={() => handleRename(contextMenu.node)} className="w-full text-
left px-3 py-1.5 text-xs text-gray-300 hover:bg-[#094771] hover:text-white">
          Rename
        </button>
        <button onClick={() => handleDelete(contextMenu.node)} className="w-full text-
left px-3 py-1.5 text-xs text-red-400 hover:bg-[#094771] hover:text-white">
          Delete
        </button>
      </div>
    ) }
  </div>
);
};

export default WorkspaceExplorer;

```

Backend: Server.js (Entry Point)

```
require("dotenv").config();
const express = require("express");
const http = require("http");
const cors = require("cors");
const { Server } = require("socket.io");

const connectDB = require("../Config/db");
const socketHandler = require("../Socket");
const { executeCode, LANGUAGES } = require("../CodeRunner");
const authRoutes = require("../Routes/authRoutes");
const fileRoutes = require("../Routes/FileRoutes");

const app = express();
app.use(cors());
app.use(express.json({ limit: '1mb' }));

// Auth routes
app.use("/api/auth", authRoutes);

// Workspace file routes
app.use("/api/workspace", fileRoutes);

// Code execution endpoint
app.post("/execute", async (req, res) => {
  const { language, code } = req.body;

  if (!language || !code) {
    return res.status(400).json({
      success: false,
      error: "Missing 'language' or 'code' in request body"
    });
  }

  try {
    const result = await executeCode(language, code);
    res.json(result);
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});

// Get supported languages
app.get("/languages", (req, res) => {
  res.json({ languages: Object.keys(LANGUAGES) });
});

const startServer = async () => {
  const dbConnected = await connectDB();

  const server = http.createServer(app);

  const io = new Server(server, {
    cors: { origin: "*" },
  });

  socketHandler(io, dbConnected);

  server.listen(3001, () => console.log("Server running on port 3001"));
};

startServer();
```

Backend: Socket.js (WebSocket Handler)

```
const Room = require("../Models/Room");
const { initTerminalHandlers } = require("../TerminalService");
const fs = require('fs');
const path = require('path');

const WORKSPACE_DIR = path.join(__dirname, 'workspace');

const inMemoryRooms = new Map();

// Generate 6-character alphanumeric Room ID
function generateRoomId() {
  const chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"; // Removed confusing chars: 0,O,I,1
  let result = "";
  for (let i = 0; i < 6; i++) {
    result += chars.charAt(Math.floor(Math.random() * chars.length));
  }
  return result;
}

function socketHandler(io, dbConnected) {
  io.on("connection", (socket) => {
    console.log("User connected:", socket.id);

    // Initialize Terminal handlers for this socket (node-pty based)
    initTerminalHandlers(socket, io);

    // === FILE SYNC ===
    socket.on('file:save', ({ filename, content }) => {
      try {
        // Basic security: prevent traversing up
        const safeName = path.basename(filename);
        const filepath = path.join(WORKSPACE_DIR, safeName);
        fs.writeFileSync(filepath, content);
        console.log(`[File] Saved ${safeName} to workspace`);
      } catch (e) {
        console.error(`[File] Error saving ${filename}:`, e.message);
      }
    });

    // === ROOM CREATION (Host Only) ===
    socket.on("create_room", async ({ userId, userName }) => {
      try {
        let roomId = generateRoomId();

        // Ensure unique room ID
        if (dbConnected) {
          let attempts = 0;
          while (await Room.findOne({ roomId }) && attempts < 10) {
            roomId = generateRoomId();
            attempts++;
          }
        }

        const room = await Room.create({
          roomId,
          hostId: userId,
          hostSocketId: socket.id,
          hostName: userName,
          code: "",
          approvedUsers: [{ odId: userId, userName, socketId: socket.id }],
        });

        socket.join(roomId);
        socket.emit("room_created", {
          roomId,
          isHost: true,
          hostName: userName
        });
        console.log(`Room ${roomId} created by ${userName}`);
      }
    });
  });
}
```

```

    } else {
      // In-memory fallback
      while (inMemoryRooms.has(roomId)) {
        roomId = generateRoomId();
      }

      inMemoryRooms.set(roomId, {
        hostSocketId: socket.id,
        hostName: userName,
        code: "",
        pendingRequests: [],
        approvedUsers: [{ userName, socketId: socket.id }],
      });

      socket.join(roomId);
      socket.emit("room_created", {
        roomId,
        isHost: true,
        hostName: userName
      });
    }
  } catch (error) {
    console.error("Create room error:", error);
    socket.emit("room_error", { error: "Failed to create room" });
  }
});

// === JOIN REQUEST (Non-Host) ===
socket.on("request_join", async ({ roomId, userId, userName }) => {
  try {
    roomId = roomId.toUpperCase();

    if (dbConnected) {
      const room = await Room.findOne({ roomId });

      if (!room) {
        socket.emit("join_error", { error: "Room not found. Please check the Room ID." });
        return;
      }

      // Add to pending requests
      room.pendingRequests.push({
        odId: userId,
        userName,
        socketId: socket.id,
      });
      await room.save();

      // Notify host
      if (room.hostSocketId) {
        io.to(room.hostSocketId).emit("join_request", {
          socketId: socket.id,
          userName,
          userId,
          roomId,
        });
      }

      socket.emit("join_pending", {
        message: "Join request sent. Waiting for host approval..."
      });
    } else {
      // In-memory fallback
      const room = inMemoryRooms.get(roomId);

      if (!room) {
        socket.emit("join_error", { error: "Room not found. Please check the Room ID." });
        return;
      }

      room.pendingRequests.push({ userName, socketId: socket.id });

      // Notify host
      io.to(room.hostSocketId).emit("join_request", {
        socketId: socket.id,
        userName,
        roomId,
      });
    }

    socket.emit("join_pending", {
      message: "Join request sent. Waiting for host approval..."
    });
  }
});

```

```

    });
  }
} catch (error) {
  console.error("Join request error:", error);
  socket.emit("join_error", { error: "Failed to send join request" });
}
});

// === HOST RESPONDS TO JOIN REQUEST ===
socket.on("join_response", async ({ roomId, requesterSocketId, accepted, userName }) => {
  try {
    roomId = roomId.toUpperCase();

    if (dbConnected) {
      const room = await Room.findOne({ roomId });

      if (!room || room.hostSocketId !== socket.id) {
        socket.emit("room_error", { error: "Only the host can approve/decline requests" });
        return;
      }

      // Remove from pending
      room.pendingRequests = room.pendingRequests.filter(
        req => req.socketId !== requesterSocketId
      );

      if (accepted) {
        // Add to approved users
        room.approvedUsers.push({ userName, socketId: requesterSocketId });
        await room.save();

        // Join the requester to the room
        const requesterSocket = io.sockets.sockets.get(requesterSocketId);
        if (requesterSocket) {
          requesterSocket.join(roomId);

          // Notify requester they're accepted
          io.to(requesterSocketId).emit("join_accepted", {
            roomId,
            code: room.code,
            hostName: room.hostName,
          });

          // Notify everyone in room about new user
          socket.to(roomId).emit("user_joined", {
            userName,
            socketId: requesterSocketId
          });
        }
      } else {
        await room.save();
        // Notify requester they're declined
        io.to(requesterSocketId).emit("join_declined", {
          message: "Your join request was declined by the host."
        });
      }
    } else {
      // In-memory
      const room = inMemoryRooms.get(roomId);

      if (!room || room.hostSocketId !== socket.id) {
        socket.emit("room_error", { error: "Only the host can approve/decline requests" });
        return;
      }

      room.pendingRequests = room.pendingRequests.filter(
        req => req.socketId !== requesterSocketId
      );

      if (accepted) {
        room.approvedUsers.push({ userName, socketId: requesterSocketId });

        const requesterSocket = io.sockets.sockets.get(requesterSocketId);
        if (requesterSocket) {
          requesterSocket.join(roomId);

          io.to(requesterSocketId).emit("join_accepted", {
            roomId,
            code: room.code,
            hostName: room.hostName,
          });
        }
      }
    }
  }
});

```

```

        socket.to(roomId).emit("user_joined", {
            userName,
            socketId: requesterSocketId
        });
    }
    } else {
        io.to(requesterSocketId).emit("join_declined", {
            message: "Your join request was declined by the host."
        });
    }
}
} catch (error) {
    console.error("Join response error:", error);
    socket.emit("room_error", { error: "Failed to process join response" });
}
});

// === HOST REMOVES USER ===
socket.on("remove_user", async ({ roomId, targetSocketId, targetUserName }) => {
    try {
        roomId = roomId.toUpperCase();

        if (dbConnected) {
            const room = await Room.findOne({ roomId });

            if (!room || room.hostSocketId !== socket.id) {
                socket.emit("room_error", { error: "Only the host can remove users" });
                return;
            }

            // Remove from approved users
            room.approvedUsers = room.approvedUsers.filter(
                u => u.socketId !== targetSocketId
            );
            await room.save();

        } else {
            const room = inMemoryRooms.get(roomId);
            if (!room || room.hostSocketId !== socket.id) {
                socket.emit("room_error", { error: "Only the host can remove users" });
                return;
            }

            room.approvedUsers = room.approvedUsers.filter(
                u => u.socketId !== targetSocketId
            );
        }

        // Kick the user from room
        const targetSocket = io.sockets.sockets.get(targetSocketId);
        if (targetSocket) {
            targetSocket.leave(roomId);
            io.to(targetSocketId).emit("user_removed", {
                message: "You have been removed from the room by the host."
            });
        }

        // Notify room
        io.to(roomId).emit("user_left", {
            userName: targetUserName,
            socketId: targetSocketId,
            removed: true
        });
    } catch (error) {
        console.error("Remove user error:", error);
        socket.emit("room_error", { error: "Failed to remove user" });
    }
});

// === LEGACY: Direct join for host after creation ===
socket.on("join_room", async ({ roomId, username }) => {
    socket.join(roomId);

    let code = "";

    if (dbConnected) {
        let room = await Room.findOne({ roomId });
        if (room) {
            code = room.code;
        }
    }

```

```

    } else {
      if (inMemoryRooms.has(roomId)) {
        code = inMemoryRooms.get(roomId).code || "";
      }
    }

    socket.emit("load_code", { code });
  });

// === CODE CHANGES ===
socket.on("code_change", async ({ roomId, code }) => {
  socket.to(roomId).emit("code_update", { code });

  if (dbConnected) {
    await Room.updateOne(
      { roomId },
      { code, updatedAt: new Date() }
    );
  } else {
    const room = inMemoryRooms.get(roomId);
    if (room) {
      room.code = code;
    }
  }
});

// === GET ROOM USERS ===
socket.on("get_room_users", async ({ roomId }) => {
  roomId = roomId.toUpperCase();

  if (dbConnected) {
    const room = await Room.findOne({ roomId });
    if (room) {
      socket.emit("room_users", {
        users: room.approvedUsers,
        hostSocketId: room.hostSocketId
      });
    }
  } else {
    const room = inMemoryRooms.get(roomId);
    if (room) {
      socket.emit("room_users", {
        users: room.approvedUsers,
        hostSocketId: room.hostSocketId
      });
    }
  }
});

// === DISCONNECT ===
socket.on("disconnect", async () => {
  console.log("User disconnected:", socket.id);

  // Handle user leaving rooms - notify others
  if (dbConnected) {
    const rooms = await Room.find({
      "approvedUsers.socketId": socket.id
    });

    for (const room of rooms) {
      const user = room.approvedUsers.find(u => u.socketId === socket.id);
      if (user) {
        io.to(room.roomId).emit("user_left", {
          userName: user.userName,
          socketId: socket.id
        });
      }
    }
  }
});
});
}

module.exports = socketHandler;

```

Backend: CodeRunner.js (Execution Logic)

```
const { exec, spawn } = require('child_process');
const fs = require('fs');
const path = require('path');
const os = require('os');
const { v4: uuidv4 } = require('uuid');

// Language configurations
const LANGUAGES = {
  javascript: {
    extension: 'js',
    command: (file) => `node "${file}"`,
    timeout: 10000
  },
  python: {
    extension: 'py',
    command: (file) => `python3 "${file}"`,
    timeout: 10000
  },
  java: {
    extension: 'java',
    compile: (file, dir) => `cd "${dir}" && javac "${path.basename(file)}"`,
    run: (file, dir) => {
      const className = path.basename(file, '.java');
      return `cd "${dir}" && java "${className}"`;
    },
    timeout: 15000
  },
  c: {
    extension: 'c',
    compile: (file, dir) => `gcc "${file}" -o "${path.join(dir, 'program')}"`,
    run: (file, dir) => `"${path.join(dir, 'program')}"`,
    timeout: 15000
  },
  cpp: {
    extension: 'cpp',
    compile: (file, dir) => `g++ "${file}" -o "${path.join(dir, 'program')}"`,
    run: (file, dir) => `"${path.join(dir, 'program')}"`,
    timeout: 15000
  },
  typescript: {
    extension: 'ts',
    // Run TypeScript using ts-node or transpile first
    command: (file) => `npx ts-node "${file}" 2>/dev/null || node "${file}"`,
    timeout: 15000
  }
};

// Execute a command with timeout
function executeCommand(command, timeout = 10000) {
  return new Promise((resolve, reject) => {
    const process = exec(command, { timeout, maxBuffer: 1024 * 1024 }, (error, stdout, stderr) => {
      if (error) {
        if (error.killed) {
          reject({ message: 'Execution timed out', stdout, stderr });
        } else {
          reject({ message: error.message, stdout, stderr });
        }
      } else {
        resolve({ stdout, stderr });
      }
    });
  });
}

// Main execution function
async function executeCode(language, code) {
  const langConfig = LANGUAGES[language];
```

```

    if (!langConfig) {
      return {
        success: false,
        error: `Language "${language}" is not supported. Supported:
${Object.keys(LANGUAGES).join(', ')}`
      };
    }

    // Create temp directory for code execution
    const tempDir = path.join(os.tmpdir(), `coscript-${uuidv4()}`);
    fs.mkdirSync(tempDir, { recursive: true });

    // Determine filename (for Java, class name must match)
    let filename;
    if (language === 'java') {
      // Extract public class name from Java code
      const classMatch = code.match(/public\s+class\s+(\w+)/);
      filename = classMatch ? `${classMatch[1]}.java` : 'Main.java';
    } else {
      filename = `code.${langConfig.extension}`;
    }

    const filePath = path.join(tempDir, filename);

    try {
      // Write code to file
      fs.writeFileSync(filePath, code);

      let result;

      if (langConfig.compile) {
        // Compiled language (Java, C++)
        try {
          // Compile
          await executeCommand(langConfig.compile(filePath, tempDir), langConfig.timeout);
          // Run
          result = await executeCommand(langConfig.run(filePath, tempDir), langConfig.timeout);
        } catch (compileError) {
          return {
            success: false,
            error: compileError.stderr || compileError.message,
            type: 'compilation'
          };
        }
      } else {
        // Interpreted language (JavaScript, Python)
        try {
          result = await executeCommand(langConfig.command(filePath), langConfig.timeout);
        } catch (runError) {
          return {
            success: false,
            error: runError.stderr || runError.message,
            type: 'runtime'
          };
        }
      }

      return {
        success: true,
        output: result.stdout || '',
        stderr: result.stderr || ''
      };
    } finally {
      // Cleanup temp directory
      try {
        fs.rmSync(tempDir, { recursive: true, force: true });
      } catch (e) {
        // Ignore cleanup errors
      }
    }
  }

  module.exports = { executeCode, LANGUAGES };

```

Backend: TerminalService.js (PTY Handling)

```
const { spawn } = require('child_process');
const os = require('os');

// Map<terminalId, { process, socketId, cwd, name, timeout }>
const terminals = new Map();
const SESSION_TIMEOUT = 5 * 60 * 1000;
let idCounter = 0;
let _io = null;

function getDefaultShell() {
  // On macOS/Linux, use bash with interactive flag for proper prompt display
  // zsh -i with pipes hangs, so bash is more reliable
  return '/bin/bash';
}

function generateId() {
  return `term_${++idCounter}_${Date.now()}`;
}

/** Emit to the correct socket for a terminal session */
function emitToSession(session, event, data) {
  if (_io && session.socketId) {
    const targetSocket = _io.sockets.sockets.get(session.socketId);
    if (targetSocket) {
      targetSocket.emit(event, data);
    }
  }
}

/**
 * Attach terminal socket handlers to a socket connection.
 * Uses child_process.spawn with bash -i for interactive mode.
 * node-pty's pty.spawn() fails on Node v24 with posix_spawn error.
 */
function initTerminalHandlers(socket, io) {
  _io = io;
  let terminalCount = 0; // Per-socket counter for naming

  // --- CREATE ---
  socket.on('terminal:create', (options = {}) => {
    const id = generateId();
    const cwd = options.cwd || process.env.HOME || os.homedir();
    terminalCount++;
    const name = options.name || `Terminal ${terminalCount}`;
    const shell = getDefaultShell();

    try {
      const proc = spawn(shell, ['--norc', '--noprofile', '-i'], {
        cwd,
        stdio: ['pipe', 'pipe', 'pipe'],
        env: {
          ...process.env,
          TERM: 'xterm-256color',
          COLORTERM: 'truecolor',
          LANG: 'en_US.UTF-8',
          PS1: `\\[\\033[01;32m\\]\\u@coscript\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ `,
        },
      });

      const session = {
        process: proc,
        socketId: socket.id,
        cwd,
        name,
        timeout: null,
      };
      terminals.set(id, session);
    }
  });
}
```

```

    // Stream stdout to client
    proc.stdout.on('data', (data) => {
        const s = terminals.get(id);
        if (s) emitToSession(s, 'terminal:output', { id, data: data.toString() });
    });

    // Stream stderr to client (bash sends prompts via stderr)
    proc.stderr.on('data', (data) => {
        const s = terminals.get(id);
        if (s) emitToSession(s, 'terminal:output', { id, data: data.toString() });
    });

    proc.on('exit', (code, signal) => {
        const s = terminals.get(id);
        if (s) emitToSession(s, 'terminal:exit', { id, exitCode: code || 0 });
        terminals.delete(id);
        console.log(`[Terminal] ${id} exited (code=${code})`);
    });

    proc.on('error', (err) => {
        console.error(`[Terminal] ${id} error:`, err.message);
        const s = terminals.get(id);
        if (s) emitToSession(s, 'terminal:error', { id, error: err.message });
        terminals.delete(id);
    });

    socket.emit('terminal:created', { id, name, shell, cwd });
    console.log(`[Terminal] Created ${id} (${shell} -i) in ${cwd}`);

    } catch (err) {
        console.error('[Terminal] Create failed:', err.message);
        socket.emit('terminal:error', { id, error: err.message });
    }
});

// --- INPUT ---
socket.on('terminal:input', ({ id, data }) => {
    const session = terminals.get(id);
    if (session && session.process && session.process.stdin && !session.process.stdin.destroyed)
{
        try {
            session.process.stdin.write(data);
        } catch (e) { }
    }
});

// --- RESIZE (no-op for child_process pipes, but accept event) ---
socket.on('terminal:resize', () => { });

// --- KILL ---
socket.on('terminal:kill', ({ id }) => {
    const session = terminals.get(id);
    if (session && session.process) {
        try {
            session.process.kill('SIGTERM');
            setTimeout(() => { try { session.process.kill('SIGKILL'); } catch (e) { } }, 1000);
        } catch (e) { }
        terminals.delete(id);
        socket.emit('terminal:killed', { id });
        console.log(`[Terminal] Killed ${id}`);
    }
});

// --- RENAME ---
socket.on('terminal:rename', ({ id, name }) => {
    const session = terminals.get(id);
    if (session) {
        session.name = name;
        socket.emit('terminal:renamed', { id, name });
    }
});

// --- LIST ---
socket.on('terminal:list', () => {
    const list = [];
    for (const [id, session] of terminals.entries()) {
        if (session.socketId === socket.id) {
            list.push({ id, name: session.name, cwd: session.cwd });
        }
    }
    socket.emit('terminal:list', list);
}

```

```
});

// --- DISCONNECT ---
socket.on('disconnect', () => {
  for (const [id, session] of terminals.entries()) {
    if (session.socketId === socket.id) {
      console.log(`[Terminal] Socket disconnected. Scheduling cleanup for ${id}`);
      session.timeout = setTimeout(() => {
        console.log(`[Terminal] Timeout. Killing ${id}`);
        try { session.process.kill(); } catch (e) { }
        terminals.delete(id);
      }, SESSION_TIMEOUT);
    }
  }
});
}

module.exports = { initTerminalHandlers };
```

Backend: authRoutes.js (Authentication Routing)

```
const express = require("express");
const User = require("../Models/User");
const { authMiddleware, generateToken } = require("../Middleware/authMiddleware");

const router = express.Router();

// Email Signup
router.post("/signup", async (req, res) => {
  try {
    const { email, password, name } = req.body;

    // Validation
    if (!email || !password || !name) {
      return res.status(400).json({
        success: false,
        error: "Email, password, and name are required",
      });
    }

    // Password length validation
    if (password.length < 6) {
      return res.status(400).json({
        success: false,
        error: "Password must be at least 6 characters",
      });
    }

    // Check if user exists
    const existingUser = await User.findOne({ email: email.toLowerCase() });
    if (existingUser) {
      return res.status(400).json({
        success: false,
        error: "An account with this email already exists",
      });
    }

    // Create user
    const user = await User.create({
      email: email.toLowerCase(),
      password,
      name,
      authProvider: "email",
    });

    const token = generateToken(user._id);

    res.status(201).json({
      success: true,
      user: {
        id: user._id,
        email: user.email,
        name: user.name,
      },
      token,
    });
  } catch (error) {
    console.error("Signup error:", error);
    res.status(500).json({
      success: false,
      error: "Failed to create account",
    });
  }
});

// Email Login
router.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;
```

```

    if (!email || !password) {
      return res.status(400).json({
        success: false,
        error: "Email and password are required",
      });
    }

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) {
      return res.status(401).json({
        success: false,
        error: "Invalid email or password",
      });
    }

    const isMatch = await user.comparePassword(password);
    if (!isMatch) {
      return res.status(401).json({
        success: false,
        error: "Invalid email or password",
      });
    }

    const token = generateToken(user._id);

    res.json({
      success: true,
      user: {
        id: user._id,
        email: user.email,
        name: user.name,
      },
      token,
    });
  } catch (error) {
    console.error("Login error:", error);
    res.status(500).json({
      success: false,
      error: "Failed to login",
    });
  }
});

// Google OAuth
router.post("/google", async (req, res) => {
  try {
    const { email, name, googleId } = req.body;

    if (!email || !name) {
      return res.status(400).json({
        success: false,
        error: "Email and name are required",
      });
    }

    let user = await User.findOne({ email: email.toLowerCase() });

    if (!user) {
      // Create new user for Google OAuth
      user = await User.create({
        email: email.toLowerCase(),
        name,
        authProvider: "google",
      });
    }

    const token = generateToken(user._id);

    res.json({
      success: true,
      user: {
        id: user._id,
        email: user.email,
        name: user.name,
      },
      token,
    });
  } catch (error) {
    console.error("Google auth error:", error);
    res.status(500).json({
      success: false,

```

```

        error: "Failed to authenticate with Google: " + error.message,
    });
}
});

// Apple OAuth
router.post("/apple", async (req, res) => {
    try {
        const { email, name, appleId } = req.body;

        if (!email || !name) {
            return res.status(400).json({
                success: false,
                error: "Email and name are required",
            });
        }

        let user = await User.findOne({ email: email.toLowerCase() });

        if (!user) {
            user = await User.create({
                email: email.toLowerCase(),
                name,
                authProvider: "apple",
            });
        }

        const token = generateToken(user._id);

        res.json({
            success: true,
            user: {
                id: user._id,
                email: user.email,
                name: user.name,
            },
            token,
        });
    } catch (error) {
        console.error("Apple auth error:", error);
        res.status(500).json({
            success: false,
            error: "Failed to authenticate with Apple",
        });
    }
});

// Get current user
router.get("/me", authMiddleware, async (req, res) => {
    res.json({
        success: true,
        user: {
            id: req.user._id,
            email: req.user.email,
            name: req.user.name,
        },
    });
});

module.exports = router;

```

Frontend: AuthPage.jsx

```
import { useState } from "react";
import { useGoogleLogin } from "@react-oauth/google";
import { useNavigate } from "react-router-dom";
import { useAuth } from "../AuthContext";
import "../AuthPage.css";

export default function AuthPage() {
  const [mode, setMode] = useState("login"); // "login" or "signup"
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [name, setName] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const { login, signup, loginWithGoogle, loginWithApple } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");

    // Password validation for signup
    if (mode === "signup" && password.length < 6) {
      setError("Password must be at least 6 characters");
      return;
    }

    setLoading(true);

    try {
      if (mode === "login") {
        await login(email, password);
      } else {
        await signup(email, password, name);
      }
      navigate("/");
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  const googleLogin = useGoogleLogin({
    onSuccess: async (tokenResponse) => {
      try {
        console.log("Google Login Success, token:", tokenResponse);
        setLoading(true);
        // 1. Get user info from Google using the access token
        const userInfoResponse = await fetch("https://www.googleapis.com/oauth2/v3/userinfo",
{
          headers: { Authorization: `Bearer ${tokenResponse.access_token}` },
        });
        const userInfo = await userInfoResponse.json();
        console.log("Google User Info:", userInfo);

        // 2. Send to backend
        await loginWithGoogle(userInfo);
        navigate("/");
      } catch (err) {
        console.error("Google login processing failed:", err);
        setError("Failed to process Google login: " + err.message);
      } finally {
        setLoading(false);
      }
    },
    onError: (errorResponse) => {
      console.error("Google Login Error:", errorResponse);
    }
  });
}
```

```

        setError("Google login popup failed/closed.");
        setLoading(false);
    },
    });

const handleGoogleLogin = () => {
    setError("");
    googleLogin();
};

const handleAppleLogin = async () => {
    // For demo purposes - in production, use Apple Sign-In SDK
    setError("Apple OAuth requires configuration. Please use email login for now.");
};

return (
    <div className="auth-container">
        <div className="auth-card">
            { /* Logo */ }
            <div className="auth-logo">
                <div className="logo-icon bg-primary text-white rounded-lg flex items-center justify-center" style={{ width: '48px', height: '48px', borderRadius: '12px' }}>
                    <span className="material-icons-round text-2xl">code</span>
                </div>
                <span className="logo-text text-2xl font-extrabold tracking-tight text-slate-900 ml-3">
                    CoScript
                </span>
            </div>

            <h1 className="auth-title">
                {mode === "login" ? "Welcome back" : "Create your account"}
            </h1>
            <p className="auth-subtitle">
                {mode === "login"
                    ? "Sign in to continue to your collaborative coding sessions"
                    : "Join CoScript to start coding together"}
            </p>

            { /* OAuth Buttons */ }
            <div className="oauth-buttons">
                <button
                    type="button"
                    className="oauth-btn google-btn"
                    onClick={handleGoogleLogin}
                >
                    <svg viewBox="0 0 24 24" width="20" height="20">
                        <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z" />
                        <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98-.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z" />
                        <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z" />
                        <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z" />
                    </svg>
                    Continue with Google
                </button>

                <button
                    type="button"
                    className="oauth-btn apple-btn"
                    onClick={handleAppleLogin}
                >
                    <svg viewBox="0 0 24 24" width="20" height="20" fill="currentColor">
                        <path d="M17.05 20.28c-.98.95-2.05.8-3.08.35-1.09-.46-2.09-.48-3.24 0-1.44.62-2.24.35-3.06-.35C2.79 15.25 3.51 7.59 9.05 7.31c1.35.07 2.29.74 3.08.8 1.18-.24 2.31-.93 3.57-.84 1.51.12 2.65.72 3.4 1.8-3.12 1.87-2.38 5.98.48 7.13-.57 1.5-1.31 2.99-2.54 4.09zM12.03 7.25c-.15-2.23 1.66-4.07 3.74-4.25.29 2.58-2.34 4.5-3.74 4.25z" />
                    </svg>
                    Continue with Apple
                </button>
            </div>

            <div className="divider">
                <span>or continue with email</span>
            </div>

            { /* Error Message */ }
            {error && (
                <div className="error-message">

```

```

        <span className="material-icons-round">error</span>
      {error}
    </div>
  )}

  {/ * Form */}
  <form onSubmit={handleSubmit} className="auth-form">
    {mode === "signup" && (
      <div className="form-group">
        <label htmlFor="name">Full Name</label>
        <input
          type="text"
          id="name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          placeholder="Enter your name"
          required
        />
      </div>
    )}

    <div className="form-group">
      <label htmlFor="email">Email Address</label>
      <input
        type="email"
        id="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="you@example.com"
        required
      />
    </div>

    <div className="form-group">
      <label htmlFor="password">Password</label>
      <input
        type="password"
        id="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder={mode === "signup" ? "Minimum 6 characters" : "Enter your
password"}
        required
      />
      {mode === "signup" && password.length > 0 && password.length < 6 && (
        <span className="password-hint error">
          Password must be at least 6 characters
        </span>
      )}
    </div>

    <button
      type="submit"
      className="submit-btn"
      disabled={loading}
    >
      {loading ? (
        <>
          <span className="spinner"></span>
          {mode === "login" ? "Signing in..." : "Creating account..."}
        </>
      ) : (
        mode === "login" ? "Sign In" : "Create Account"
      )}
    </button>
  </form>

  {/ * Toggle Mode */}
  <p className="toggle-mode">
    {mode === "login" ? (
      <>
        Don't have an account?{" "}
        <button type="button" onClick={() => { setMode("signup"); setError(""); }}
        Sign up
      </button>
    </>
      ) : (
        <>
          Already have an account?{" "}
          <button type="button" onClick={() => { setMode("login"); setError(""); }}
        </button>
      </>
    )}
  </p>

```

```
                Sign in
            </button>
        </>
    })
</p>
</div>

    { /* Background decoration */}
    <div className="auth-bg-decoration">
        <div className="bg-circle circle-1"></div>
        <div className="bg-circle circle-2"></div>
        <div className="bg-circle circle-3"></div>
    </div>
</div>
    );
}
```

Frontend: Home.jsx

```
import { useNavigate } from "react-router-dom";
import { useState, useEffect } from "react";
import { socket } from "../socket";
import { useAuth } from "../AuthContext";
import "../Home.css";

export default function Home() {
  const [roomId, setRoomId] = useState("");
  const [userName, setUserName] = useState("");
  const [joinStatus, setJoinStatus] = useState(""); // "", "pending", "declined"
  const [createdRoomId, setCreatedRoomId] = useState("");
  const [copied, setCopied] = useState(false);
  const [timeLeft, setTimeLeft] = useState({ days: 0, hours: 0, minutes: 0, seconds: 0 });

  const navigate = useNavigate();
  const { user, logout } = useAuth();

  // Set username from auth
  useEffect(() => {
    if (user?.name) {
      setUserName(user.name);
    }
  }, [user]);

  // Countdown timer
  useEffect(() => {
    const targetDate = new Date('February 28, 2026 12:00:00 GMT+05:30');

    const timer = setInterval(() => {
      const now = new Date();
      const nowIST = new Date(now.getTime() + (5.5 * 60 * 60 * 1000));
      const distance = targetDate - nowIST;

      if (distance < 0) {
        clearInterval(timer);
        setTimeLeft({ days: 0, hours: 0, minutes: 0, seconds: 0 });
        return;
      }

      setTimeLeft({
        days: Math.floor(distance / (1000 * 60 * 60 * 24)),
        hours: Math.floor((distance % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60)),
        minutes: Math.floor((distance % (1000 * 60 * 60)) / (1000 * 60)),
        seconds: Math.floor((distance % (1000 * 60)) / 1000)
      });
    }, 1000);

    return () => clearInterval(timer);
  }, []);

  // Socket event handlers
  useEffect(() => {
    // Room created successfully
    socket.on("room_created", ({ roomId, isHost }) => {
      setCreatedRoomId(roomId);
      // Navigate to the room as host
      navigate(`/room/${roomId}?host=true`);
    });

    // Join request pending
    socket.on("join_pending", ({ message }) => {
      setJoinStatus("pending");
    });

    // Join accepted
    socket.on("join_accepted", ({ roomId }) => {
      setJoinStatus("");
      navigate(`/room/${roomId}`);
    });
  }, []);
}
```

```

});

// Join declined
socket.on("join_declined", ({ message }) => {
  setJoinStatus("declined");
  setTimeout(() => setJoinStatus(""), 3000);
});

// Join error
socket.on("join_error", ({ error }) => {
  setJoinStatus("");
  alert(error);
});

return () => {
  socket.off("room_created");
  socket.off("join_pending");
  socket.off("join_accepted");
  socket.off("join_declined");
  socket.off("join_error");
};
}, [navigate]);

const createRoom = () => {
  socket.emit("create_room", {
    userId: user?.id,
    userName: user?.name || "Host"
  });
};

const joinRoom = () => {
  if (!roomId.trim()) return;
  if (!userName.trim()) {
    alert("Please enter your name");
    return;
  }

  socket.emit("request_join", {
    roomId: roomId.toUpperCase(),
    userId: user?.id,
    userName
  });
};

const handleKeyPress = (e) => {
  // if (e.key === 'Enter') {
  //   joinRoom();
  // }
};

// Dark mode hook - basic check
useEffect(() => {
  if (window.matchMedia && window.matchMedia('(prefers-color-scheme: dark)').matches) {
    document.documentElement.classList.add('dark');
  }
}, []);

return (
  <div className="font-display text-slate-900 dark:text-slate-100 mesh-gradient min-h-screen
transition-colors duration-300">
    <nav className="sticky top-0 z-50 glass border-b border-slate-200/50 dark:border-slate-800/50">
      <div className="max-w-7xl mx-auto px-6 h-20 flex items-center justify-between">
        <div className="flex items-center gap-2">
          <div className="w-10 h-10 bg-primary rounded-xl flex items-center justify-center text-
white">
            <span className="material-icons-round">code</span>
          </div>
          <span className="text-2xl font-extrabold tracking-tight text-slate-900 dark:text-
white">CoScript</span>
        </div>
        <div className="flex items-center gap-6">
          {user ? (
            <div className="hidden md:flex flex-col items-end mr-2">
              <span className="text-xs font-medium text-slate-500 dark:text-slate-400 uppercase
tracking-wider">Signed in as</span>
              <span className="text-sm font-semibold">{user.name}</span>
            </div>
          ) : null}

          {user && (
            <button
              onClick={logout}

```

```

        className="px-5 py-2.5 bg-slate-100 hover:bg-slate-200 dark:bg-slate-800
dark: hover:bg-slate-700 text-slate-700 dark:text-slate-200 font-semibold rounded-lg transition-all
flex items-center gap-2"
    >
        <span className="material-icons-round text-sm">logout</span>
        Sign Out
    </button>
    </div>
</div>
</nav>

<main className="max-w-7xl mx-auto px-6 pt-12 pb-24">
    <div className="text-center space-y-6 mb-16">
        <div className="inline-flex items-center gap-2 px-4 py-2 bg-blue-50 dark:bg-blue-900/30
text-primary dark:text-blue-400 rounded-full text-sm font-bold tracking-wide uppercase">
            <span className="relative flex h-2 w-2">
                <span className="animate-ping absolute inline-flex h-full w-full rounded-full bg-blue-
400 opacity-75"></span>
                <span className="relative inline-flex rounded-full h-2 w-2 bg-blue-500"></span>
            </span>
            Now Live: BHAVANS Edition
        </div>
        <h1 className="text-5xl md:text-7xl font-extrabold tracking-tight text-slate-900 dark:text-
white leading-[1.1]">
            Collaborative <span className="text-transparent bg-clip-text bg-gradient-to-r from-
primary to-emerald-500">Code Editor</span>
        </h1>
        <p className="text-lg md:text-xl text-slate-600 dark:text-slate-400 max-w-2xl mx-auto font-
medium">
            Discover, code, collaborate - CoScript makes real-time coding effortless. Your next
            project starts right here.
        </p>
        <div className="flex flex-wrap justify-center gap-3 pt-4">
            <div className="flex items-center gap-2 px-4 py-2 bg-white/50 dark:bg-slate-800/50 border
border-slate-200 dark:border-slate-700 rounded-full shadow-sm">
                <span className="material-icons-round text-emerald-500 text-lg">check_circle</span>
                <span className="text-sm font-semibold">Real-time collaboration</span>
            </div>
            <div className="flex items-center gap-2 px-4 py-2 bg-white/50 dark:bg-slate-800/50 border
border-slate-200 dark:border-slate-700 rounded-full shadow-sm">
                <span className="material-icons-round text-emerald-500 text-lg">check_circle</span>
                <span className="text-sm font-semibold">Multiple languages</span>
            </div>
            <div className="flex items-center gap-2 px-4 py-2 bg-white/50 dark:bg-slate-800/50 border
border-slate-200 dark:border-slate-700 rounded-full shadow-sm">
                <span className="material-icons-round text-emerald-500 text-lg">check_circle</span>
                <span className="text-sm font-semibold">Instant sync</span>
            </div>
        </div>
        <div className="relative max-w-4xl mx-auto mb-20 group">
            <div className="absolute -inset-4 bg-gradient-to-r from-primary to-emerald-500 rounded-
[2.5rem] opacity-20 blur-2xl group-hover:opacity-30 transition-opacity"></div>
            <div className="relative glass rounded-2xl overflow-hidden shadow-2xl border border-
white/20 dark:border-slate-700/50">
                <div className="bg-slate-100/80 dark:bg-slate-800/80 px-4 py-3 flex items-center justify-
between border-b border-slate-200 dark:border-slate-700">
                    <div className="flex gap-1.5">
                        <div className="w-3 h-3 rounded-full bg-red-400"></div>
                        <div className="w-3 h-3 rounded-full bg-yellow-400"></div>
                        <div className="w-3 h-3 rounded-full bg-green-400"></div>
                    </div>
                    <div className="flex items-center gap-2 px-3 py-1 bg-white dark:bg-slate-900 rounded-md
shadow-sm">
                        <span className="material-icons-round text-sm text-slate-400">description</span>
                        <span className="text-xs font-mono text-slate-600 dark:text-slate-300">main.js</span>
                    </div>
                    <div className="flex -space-x-2">
                        <div className="w-7 h-7 rounded-full ring-2 ring-white dark:ring-slate-800 bg-blue-
500 flex items-center justify-center text-[10px] text-white font-bold">SM</div>
                        <div className="w-7 h-7 rounded-full ring-2 ring-white dark:ring-slate-800 bg-
emerald-500 flex items-center justify-center text-[10px] text-white font-bold">JD</div>
                        <div className="w-7 h-7 rounded-full ring-2 ring-white dark:ring-slate-800 bg-amber-
500 flex items-center justify-center text-[10px] text-white font-bold">AL</div>
                    </div>
                </div>
                <div className="p-8 font-mono text-sm leading-relaxed overflow-x-auto bg-slate-50
dark:bg-[#0b1222]">
                    <div className="flex gap-6">
                        <div className="text-slate-400 text-right select-none space-y-0.5">

```

```

    <div>1</div><div>2</div><div>3</div><div>4</div><div>5</div><div>6</div>
<div>7</div><div>8</div>
  </div>
  <div className="space-y-0.5">
    <div className="text-purple-500 dark:text-purple-400">function <span
className="text-blue-600 dark:text-blue-400">collaborate</span>() {'{'</div>
    <div className="pl-4"><span className="text-slate-500 dark:text-slate-400">//
Connecting peers...</span></div>
    <div className="pl-4">
      <span className="text-slate-900 dark:text-slate-100">const room = </span>
      <span className="text-emerald-600 dark:text-emerald-400">"CO-SCRIPT-PRO"</span>;
    </div>
    <div className="pl-4 flex items-center">
      <span className="text-slate-900 dark:text-slate-100">console.log(</span>
      <span className="text-emerald-600 dark:text-emerald-400">"Starting session"
    </span>
    <span className="text-slate-900 dark:text-slate-100">);</span>
    <div className="relative ml-1">
      <div className="w-0.5 h-5 bg-blue-500 cursor-blink"></div>
      <div className="absolute -top-6 left-0 px-1.5 py-0.5 bg-blue-500 text-white
text-[10px] rounded font-sans font-bold whitespace-nowrap">Sujal</div>
    </div>
    <div className="pl-4"><span className="text-slate-900 dark:text-slate-100">return
true;</span></div>
    <div>{'{'</div>
    <div className="pt-2 flex items-center">
      <span className="text-blue-600 dark:text-blue-400">collaborate</span>
      <span className="text-slate-900 dark:text-slate-100">();</span>
      <div className="relative ml-1">
        <div className="w-0.5 h-5 bg-emerald-500"></div>
        <div className="absolute -bottom-6 left-0 px-1.5 py-0.5 bg-emerald-500 text-
white text-[10px] rounded font-sans font-bold whitespace-nowrap">Sarah</div>
      </div>
    </div>
  </div>
</div>
</div>
</div>

  <div className="max-w-2xl mx-auto glass rounded-[2rem] p-8 md:p-12 shadow-2xl border border-
white/30 dark:border-slate-700/30" id="actions">
    <h2 className="text-2xl font-extrabold text-center mb-10 tracking-tight text-slate-900
dark:text-white">CREATE OR JOIN ROOM</h2>
    <div className="flex flex-col gap-8">
      <button
        onClick={createRoom}
        className="w-full bg-primary hover:bg-blue-700 text-white py-5 px-8 rounded-2xl font-
bold text-lg shadow-lg shadow-blue-500/25 transition-all active:scale-[0.98] flex items-center
justify-center gap-3">
        >
        <span className="material-icons-round">add_circle</span>
        Create New Collaborative Room
      </button>
      <div className="relative flex items-center gap-4">
        <div className="flex-grow h-px bg-slate-200 dark:bg-slate-800"></div>
        <span className="text-xs font-bold text-slate-400 uppercase tracking-widest">or join
existing</span>
        <div className="flex-grow h-px bg-slate-200 dark:bg-slate-800"></div>
      </div>
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        <div className="relative group">
          <span className="absolute left-4 top-1/2 -translate-y-1/2 material-icons-round text-
slate-400 group-focus-within:text-primary transition-colors">vpn_key</span>
          <input
            value={roomId}
            onChange={(e) => setRoomId(e.target.value)}
            className="w-full pl-12 pr-4 py-4 bg-white/50 dark:bg-slate-900/50 border border-
slate-200 dark:border-slate-800 rounded-xl focus:ring-2 focus:ring-primary focus:border-primary
outline-none transition-all font-mono tracking-widest text-slate-900 dark:text-white"
            placeholder="Enter 6-char Code"
            type="text"
            maxLength={6}
          />
        </div>
        <button
          onClick={joinRoom}
          className="w-full bg-white dark:bg-slate-800 hover:bg-slate-50 dark:hover:bg-slate-
750 text-slate-900 dark:text-white py-4 px-6 border border-slate-200 dark:border-slate-700 rounded-xl
font-bold shadow-sm transition-all flex items-center justify-center gap-2">
          >

```

```

        {joinStatus === "pending" ? (
            <span className="material-icons-round animate-spin">refresh</span>
        ) : (
            <span className="material-icons-round">login</span>
        )}
        {joinStatus === "pending" ? "Waiting..." : "Request to Join"}
    </button>
</div>
</div>
<div className="mt-12 text-center">
    <p className="text-xs font-bold text-slate-400 dark:text-slate-500 uppercase tracking-[0.2em] mb-4">Final Release Countdown</p>
    <div className="flex justify-center gap-4 md:gap-8">
        <div className="text-center">
            <div className="text-3xl font-extrabold text-primary">
                {timeLeft.days.toString().padStart(3, '0')}</div>
            <div className="text-[10px] font-bold text-slate-400 uppercase tracking-widest">Days</div>
        </div>
        <div className="text-center">
            <div className="text-3xl font-extrabold text-primary">
                {timeLeft.hours.toString().padStart(2, '0')}</div>
            <div className="text-[10px] font-bold text-slate-400 uppercase tracking-widest">Hours</div>
        </div>
        <div className="text-center">
            <div className="text-3xl font-extrabold text-primary">
                {timeLeft.minutes.toString().padStart(2, '0')}</div>
            <div className="text-[10px] font-bold text-slate-400 uppercase tracking-widest">Mins</div>
        </div>
    </div>
    <p className="mt-6 text-sm italic text-slate-400">Until February 28, 2026 • 12:00 PM</p>
</div>
</main>

    <footer className="max-w-7xl mx-auto px-6 py-12 border-t border-slate-200/50 dark:border-slate-800/50 text-center text-slate-500 text-sm">
        <p>© 2024 CoScript Engine. Designed for performance and collaboration.</p>
    </footer>
</div>
);
}

```


11. TESTING

A process of executing a program with the explicit intention of finding errors, that is making the program fail.

TESTING METHODOLOGIES:

- Black Box Testing
- Software Testing
- White Box Testing

SOFTWARE TESTING:

It is the process of testing the functionality and correctness of software by running it. A good test case is one that has a high probability of finding a yet undiscovered error.

BLACK BOX TESTING:

Applies to software system or module, tests functionality in terms of inputs and outputs at interfaces.

WHITE/GLASS BOX TESTING:

Knowing the internal working i.e., to test if all internal operations are performed according to the program structure.

UNIT TESTING:

Using the component level design description as a guide, important control paths are tested to uncover errors within the boundary of the module.

12. IMPLEMENTATION

Implementation is the process of executing and deploying the designed system into a real-time working environment. To implement CoScript, the following operations are performed:

1. The platform is compatible with laptops and desktop computers using a modern web browser.
2. A user installs Node.js and runs the standard dual CLI sequences (`npm install`).
3. The Vite bundler transpiles React code into optimized build artifacts.
4. The MongoDB cluster string is injected via `.env` file into the Mongoose driver.
5. The local instance is hosted asynchronously via Concurrently execution.
6. Users visit `localhost:3000` to interact with the GUI while Socket.io hooks bind silently across port `3001`.

13. MAINTENANCE

It is feasible to create a system that satisfies the current requirements of the users; however, the needs of the users may change over time as the usage of the platform increases. CoScript will need to be maintained periodically to accommodate changing requirements and ensure continued efficiency and effectiveness.

With the advancement of web technologies, CoScript can be developed to incorporate new functionalities. Security is an important consideration. With the advancement of cybersecurity threats, features such as JWT authentication, executing sandboxes, and Express middleware limits will be updated.

Future goals include distributing terminal generation to isolated Docker containers across a load-balanced network to prevent system failure during mass socket emission sequences. The React-DOM state variables will be optimized using Memoization patterns to reduce unnecessary re-rendering during collaborative typing loads.

14. CONCLUSION

In summary, CoScript is an innovative solution for logical coding and algorithm synchronization that incorporates contemporary full-stack technology into a single creative platform. CoScript is developed using MERN stack architecture and Socket technology, ensuring a scalable, secure, and friendly interface for developers.

Using CoScript's friendly interface, easy navigation, and organized workspace tree management, teams can easily organize their logic flow. The application provides secure login access using Google OAuth and scalable data processing natively embedded into the browser DOM matrix.

15. BIBLIOGRAPHY

Sites-Referred:

- <https://reactjs.org/>
- <https://socket.io/>
- <https://expressjs.com/>
- <https://www.mongodb.com/>

16. APPENDIX A: SERVER LOGS

Examples of standard I/O traces spanning concurrent connections.

17. GLOSSARY

WebSocket: A computer communications protocol, providing full-duplex communication channels over a single TCP connection.

Virtual DOM: A programming concept where an ideal, or "virtual", representation of a UI is kept in memory and synced with the "real" DOM.

18. LICENSING

MIT License

